

Robotic Hand Training and R1 MuJoCo Integration Manual

Cesar A. Pascal Vasquez

Week 19

Purpose of this Manual

This manual explains how to set up, train, evaluate, and visualize a MuJoCo robotic hand policy using Gymnasium-Robotics and Stable-Baselines3. It also explains how to attach a simple articulated hand to a Unitree R1 MuJoCo model for testing. The goal is to provide enough information for another person to reproduce a working robotic hand model from the beginning.

Contents

1	Project Overview	3
2	Recommended Folder Structure	4
3	Package Installation	4
3.1	Optional Conda Environment	4
3.2	Install Required Packages	5
4	Checking Available Hand Environments	5
5	Testing the Basic Hand Environment	6
6	Training the Basic Hand Reach Policy	7
7	Playing the Basic Hand Reach Policy	9
8	Training the Advanced Block Manipulation Policy	10

9	Playing the Block Manipulation Policy	12
10	Evaluating All Saved Checkpoints	13
11	Selecting the Best Model	17
12	Continuing Training from the Best Checkpoint	18
13	Using TensorBoard	20
14	Finding the R1 XML File	21
15	Inspecting R1 Body Names	21
16	Inspecting the Original R1 Right Wrist	22
17	Creating an R1 Model with a Replacement Hand	24
18	Testing the R1 With the Replacement Hand	29
19	Adjusting Hand Position and Orientation	31
20	Troubleshooting	32
20.1	Deprecated Environment Error	32
20.2	HER Loading Error	32
20.3	R1 Falls in the Viewer	33
20.4	No Floor Appears	33
20.5	Hand Appears on the Side	33
21	Recommended Next Steps	33
22	Final Checklist	34
23	Conclusion	35

1 Project Overview

The work is divided into two main parts. The first part trains a robotic hand by itself using the Shadow Dexterous Hand environments in Gymnasium-Robotics. The first training task is a reaching task, and the second task is a more advanced block manipulation task. The second part attaches a simple robotic hand model to the Unitree R1 model in MuJoCo. This allows the hand to be tested as part of the robot body before creating a full R1 arm-and-hand training task.

The main tools used are:

- Python
- MuJoCo
- Gymnasium
- Gymnasium-Robotics
- Stable-Baselines3
- Soft Actor-Critic (SAC)
- Hindsight Experience Replay (HER)

The final workflow is:

1. Install the required packages.
2. Confirm that the Shadow Hand environments work.
3. Train a simple hand reaching policy.
4. Train a more advanced block manipulation policy.
5. Evaluate all saved checkpoints.
6. Select the best checkpoint based on success rate and mean reward.
7. Attach a simple articulated hand to the R1 right wrist in MuJoCo.
8. Test the R1 with the new hand while disabling gravity for visualization.

2 Recommended Folder Structure

The following folder structure keeps the project organized:

```
Desktop/  
|-- train_hand_reach.py  
|-- play_hand_reach.py  
|-- train_hand_block.py  
|-- play_hand_block.py  
|-- evaluate_hand_block_models.py  
|-- continue_hand_block.py  
|-- inspect_r1_bodies.py  
|-- inspect_right_wrist.py  
|-- make_r1_replacement_hand.py  
|-- test_r1_with_hand.py  
|-- logs/  
|   |-- hand_reach/  
|   |-- hand_block/  
|   |-- hand_block_continue/  
|-- models/  
|   |-- hand_reach/  
|   |-- hand_block/  
|   |-- hand_block_continue/
```

The R1 MuJoCo XML file used in this project was located at:

```
/Users/cesarpascal/Desktop/Embodie AI Reports/r1_fixed/r1_fixed.xml
```

If another computer is used, this path must be changed to match the location of the R1 XML file.

3 Package Installation

The project can be run from the base Anaconda environment or from a separate conda environment. A separate environment is recommended because it avoids package conflicts.

3.1 Optional Conda Environment

```
conda create -n mujoco_hand python=3.11 -y  
conda activate mujoco_hand
```

Python 3.11 is recommended because many robotics and reinforcement learning packages are usually stable with it. Python 3.12 may also work if the packages install correctly.

3.2 Install Required Packages

```
pip install mujoco gymnasium gymnasium-robotics stable-baselines3
tensorboard tqdm rich
```

If training uses a Mac without a GPU, force the Stable-Baselines3 models to use the CPU by adding:

```
device="cpu"
```

inside the model definition.

4 Checking Available Hand Environments

Before training, check which hand environments are installed. Create a file named `list_hand_envs.py`.

```
import gymnasium as gym
import gymnasium_robotics

gym.register_envs(gymnasium_robotics)

print("\nInstalled hand environments:\n")

for env_id in sorted(gym.registry.keys()):
    if "Hand" in env_id:
        print(env_id)
```

Listing 1: Listing all installed hand environments

Run it with:

```
python list_hand_envs.py
```

Useful environments include:

- HandReachDense-v3
- HandManipulateBlockRotateZDense-v1
- HandManipulateBlockRotateParallelDense-v1

- HandManipulateBlockRotateXYZDense-v1
- HandManipulateBlockFullDense-v1

The easiest starting point is HandReachDense-v3. The easiest advanced manipulation environment is usually HandManipulateBlockRotateZDense-v1.

5 Testing the Basic Hand Environment

Create a file called `test_hand_reach.py`. This script opens the Shadow Hand reaching environment and applies random actions to confirm that the viewer works.

```
import gymnasium as gym
import gymnasium_robotics

gym.register_envs(gymnasium_robotics)

ENV_ID = "HandReachDense-v3"

env = gym.make(ENV_ID, render_mode="human")
obs, info = env.reset()

for _ in range(1000):
    action = env.action_space.sample()
    obs, reward, terminated, truncated, info = env.step(action)

    if terminated or truncated:
        obs, info = env.reset()

env.close()
```

Listing 2: Testing the Shadow Hand reach environment

Run it with:

```
python test_hand_reach.py
```

If the MuJoCo viewer opens and the hand moves randomly, the environment is working.

6 Training the Basic Hand Reach Policy

The first training task teaches the hand to move its fingers toward target positions. This is easier than object manipulation and is useful for confirming that the reinforcement learning setup works.

Create a file called `train.hand_reach.py`.

```
import os
import gymnasium as gym
import gymnasium_robotics

from stable_baselines3 import SAC
from stable_baselines3.common.monitor import Monitor
from stable_baselines3.common.callbacks import CheckpointCallback,
    EvalCallback

gym.register_envs(gymnasium_robotics)

ENV_ID = "HandReachDense-v3"
LOG_DIR = "logs/hand_reach"
MODEL_DIR = "models/hand_reach"

os.makedirs(LOG_DIR, exist_ok=True)
os.makedirs(MODEL_DIR, exist_ok=True)

def make_env(render_mode=None):
    env = gym.make(ENV_ID, render_mode=render_mode)
    env = Monitor(env)
    return env

env = make_env()
eval_env = make_env()

checkpoint_callback = CheckpointCallback(
    save_freq=25000,
    save_path=MODEL_DIR,
    name_prefix="hand_reach_checkpoint"
```

```

)

eval_callback = EvalCallback(
    eval_env,
    best_model_save_path=MODEL_DIR,
    log_path=LOG_DIR,
    eval_freq=10000,
    deterministic=True,
    render=False
)

model = SAC(
    "MultiInputPolicy",
    env,
    learning_rate=3e-4,
    buffer_size=300000,
    batch_size=256,
    tau=0.005,
    gamma=0.95,
    train_freq=1,
    gradient_steps=1,
    learning_starts=5000,
    verbose=1,
    tensorboard_log=LOG_DIR,
    device="cpu"
)

model.learn(
    total_timesteps=200000,
    callback=[checkpoint_callback, eval_callback],
    progress_bar=True
)

final_path = os.path.join(MODEL_DIR, "hand_reach_final")
model.save(final_path)

env.close()
eval_env.close()

```

```
print(f"Saved final model to: {final_path}.zip")
```

Listing 3: Training a Shadow Hand reaching policy with SAC

Run the training with:

```
python train_hand_reach.py
```

The model files will be saved in:

```
models/hand_reach/
```

7 Playing the Basic Hand Reach Policy

Create a file called `play_hand_reach.py`.

```
import gymnasium as gym
import gymnasium_robotics

from stable_baselines3 import SAC

gym.register_envs(gymnasium_robotics)

ENV_ID = "HandReachDense-v3"
MODEL_PATH = "models/hand_reach/hand_reach_final.zip"

env = gym.make(ENV_ID, render_mode="human")
model = SAC.load(MODEL_PATH, device="cpu")

obs, info = env.reset()

while True:
    action, _ = model.predict(obs, deterministic=True)
    obs, reward, terminated, truncated, info = env.step(action)

    if terminated or truncated:
        obs, info = env.reset()
```

Listing 4: Playing the trained hand reach policy

Run it with:

```
python play_hand_reach.py
```

8 Training the Advanced Block Manipulation Policy

After the reaching task works, the next step is block manipulation. The environment used for this project was:

```
HandManipulateBlockRotateZDense-v1
```

This task is harder because the hand must rotate a block around the vertical axis. SAC is used again, but this time with Hindsight Experience Replay (HER), which helps goal-conditioned robotic tasks learn more effectively.

Create a file called `train_hand_block.py`.

```
import os
import gymnasium as gym
import gymnasium_robotics

from stable_baselines3 import SAC
from stable_baselines3.her.her_replay_buffer import HerReplayBuffer
from stable_baselines3.common.monitor import Monitor
from stable_baselines3.common.callbacks import CheckpointCallback,
    EvalCallback

gym.register_envs(gymnasium_robotics)

ENV_ID = "HandManipulateBlockRotateZDense-v1"
LOG_DIR = "logs/hand_block"
MODEL_DIR = "models/hand_block"

os.makedirs(LOG_DIR, exist_ok=True)
os.makedirs(MODEL_DIR, exist_ok=True)

def make_env(render_mode=None):
    env = gym.make(ENV_ID, render_mode=render_mode)
    env = Monitor(env)
    return env
```

```

env = make_env()
eval_env = make_env()

checkpoint_callback = CheckpointCallback(
    save_freq=50000,
    save_path=MODEL_DIR,
    name_prefix="hand_block_checkpoint"
)

eval_callback = EvalCallback(
    eval_env,
    best_model_save_path=MODEL_DIR,
    log_path=LOG_DIR,
    eval_freq=25000,
    deterministic=True,
    render=False
)

model = SAC(
    "MultiInputPolicy",
    env,
    learning_rate=3e-4,
    buffer_size=500000,
    batch_size=256,
    tau=0.005,
    gamma=0.95,
    train_freq=1,
    gradient_steps=1,
    learning_starts=10000,
    replay_buffer_class=HerReplayBuffer,
    replay_buffer_kwargs=dict(
        n_sampled_goal=4,
        goal_selection_strategy="future"
    ),
    verbose=1,
    tensorboard_log=LOG_DIR,

```

```

        device="cpu"
    )

    model.learn(
        total_timesteps=500000,
        callback=[checkpoint_callback, eval_callback],
        progress_bar=True
    )

    final_path = os.path.join(MODEL_DIR, "hand_block_final")
    model.save(final_path)

    env.close()
    eval_env.close()

    print(f"Saved final model to: {final_path}.zip")

```

Listing 5: Training the hand to rotate a block with SAC and HER

Run the training with:

```
python train_hand_block.py
```

The training will create files such as:

```

models/hand_block/hand_block_checkpoint_50000_steps.zip
models/hand_block/hand_block_checkpoint_100000_steps.zip
models/hand_block/hand_block_checkpoint_150000_steps.zip
models/hand_block/hand_block_checkpoint_200000_steps.zip
models/hand_block/hand_block_checkpoint_250000_steps.zip
models/hand_block/hand_block_checkpoint_300000_steps.zip
models/hand_block/hand_block_final.zip
models/hand_block/best_model.zip

```

9 Playing the Block Manipulation Policy

Because the model was trained with HER, the environment must be passed when loading the model. If this is not done, Stable-Baselines3 may give this error:

```

AssertionError: You must pass an environment when using '
    HerReplayBuffer

```

Create a file called `play_hand_block.py`.

```
import gymnasium as gym
import gymnasium_robotics

from stable_baselines3 import SAC

gym.register_envs(gymnasium_robotics)

ENV_ID = "HandManipulateBlockRotateZDense-v1"
MODEL_PATH = "models/hand_block/hand_block_checkpoint_300000_steps.
    zip"

env = gym.make(ENV_ID, render_mode="human")

# Important: pass env=env because this model was trained with HER.
model = SAC.load(MODEL_PATH, env=env, device="cpu")

obs, info = env.reset()

while True:
    action, _ = model.predict(obs, deterministic=True)
    obs, reward, terminated, truncated, info = env.step(action)

    if terminated or truncated:
        obs, info = env.reset()
```

Listing 6: Playing the trained block manipulation policy

Run it with:

```
python play_hand_block.py
```

10 Evaluating All Saved Checkpoints

The final model is not always the best model. For example, in this project the 300,000-step checkpoint performed better than the final model in one evaluation. Therefore, all checkpoints should be tested.

Create a file called `evaluate_hand_block_models.py`.

```

import os
import glob
import csv
import gymnasium as gym
import gymnasium_robotics

from stable_baselines3 import SAC

gym.register_envs(gymnasium_robotics)

ENV_ID = "HandManipulateBlockRotateZDense-v1"
MODEL_DIR = "models/hand_block"
RESULTS_FILE = "hand_block_evaluation_results.csv"

N_EPISODES = 20

def evaluate_model(model_path):
    env = gym.make(ENV_ID)

    # Important because the model was trained with HER.
    model = SAC.load(model_path, env=env, device="cpu")

    episode_rewards = []
    episode_successes = []

    for episode in range(N_EPISODES):
        obs, info = env.reset()

        total_reward = 0.0
        success_seen = 0.0

        terminated = False
        truncated = False

        while not (terminated or truncated):
            action, _ = model.predict(obs, deterministic=True)
            obs, reward, terminated, truncated, info = env.step(

```

```

        action)

        total_reward += reward

        if "is_success" in info:
            success_seen = max(success_seen, float(info["
                is_success"]))

        episode_rewards.append(total_reward)
        episode_successes.append(success_seen)

    env.close()

    mean_reward = sum(episode_rewards) / len(episode_rewards)
    success_rate = sum(episode_successes) / len(episode_successes)

    return mean_reward, success_rate

model_paths = sorted(glob.glob(os.path.join(MODEL_DIR, "*.zip")))

if not model_paths:
    raise FileNotFoundError(f"No .zip models found in {MODEL_DIR}")

results = []

print("\nEvaluating models...\n")

for path in model_paths:
    name = os.path.basename(path)

    try:
        mean_reward, success_rate = evaluate_model(path)

        results.append({
            "model": name,
            "path": path,
            "mean_reward": mean_reward,

```

```

        "success_rate": success_rate,
    })

    print(f"{name}")
    print(f"    Mean reward: {mean_reward:.3f}")
    print(f"    Success rate: {success_rate * 100:.1f}%\n")

except Exception as e:
    print(f"Could not evaluate {name}: {e}\n")

# Sort by success rate first, then reward.
results.sort(
    key=lambda x: (x["success_rate"], x["mean_reward"]),
    reverse=True
)

with open(RESULTS_FILE, "w", newline="") as f:
    writer = csv.DictWriter(
        f,
        fieldnames=["model", "path", "mean_reward", "success_rate"]
    )
    writer.writeheader()
    writer.writerows(results)

best = results[0]

print("=" * 60)
print("BEST MODEL")
print("=" * 60)
print(f"Model:          {best['model']}")
print(f"Path:           {best['path']}")
print(f"Mean reward:    {best['mean_reward']:.3f}")
print(f"Success rate:   {best['success_rate'] * 100:.1f}%")
print("=" * 60)

print(f"\nSaved full evaluation table to: {RESULTS_FILE}")

```

Listing 7: Evaluating all saved hand block models

Run the evaluation with:

```
python evaluate_hand_block_models.py
```

11 Selecting the Best Model

A good model should be selected using the following priority:

1. Highest success rate.
2. Highest mean reward when success rates are similar.
3. Best visual behavior in the MuJoCo viewer.

In this project, one evaluation gave the following best result:

```
BEST MODEL
=====
Model:          hand_block_checkpoint_300000_steps.zip
Path:           models/hand_block/hand_block_checkpoint_300000_steps.
               zip
Mean reward:    -99.298
Success rate:   45.0%
=====
```

A later training evaluation reached:

```
Eval num_timesteps=500000, episode_reward=-35.99 +/- 38.63
Episode length: 100.00 +/- 0.00
Success rate: 60.00%
```

This shows that the model improved during training, but checkpoints should still be tested because the best visual result may not always be the final saved model.

Table 1: Example checkpoint evaluation results

Model	Mean Reward	Success Rate
hand_block_checkpoint_300000_steps.zip	-99.30	45%
hand_block_checkpoint_250000_steps.zip	-139.39	45%
hand_block_final.zip	-90.08	40%
hand_block_checkpoint_450000_steps.zip	-112.45	40%
hand_block_checkpoint_400000_steps.zip	-119.66	40%
hand_block_checkpoint_500000_steps.zip	-83.86	35%

To save the selected checkpoint as the preferred model, run:

```
cp models/hand_block/hand_block_checkpoint_300000_steps.zip models/
hand_block/selected_best_hand_block.zip
```

Then use this path in the playback file:

```
MODEL_PATH = "models/hand_block/selected_best_hand_block.zip"
```

12 Continuing Training from the Best Checkpoint

Instead of restarting training from zero, training can continue from the best checkpoint. Create a file called `continue_hand_block.py`.

```
import os
import gymnasium as gym
import gymnasium_robotics

from stable_baselines3 import SAC
from stable_baselines3.her.her_replay_buffer import HerReplayBuffer
from stable_baselines3.common.monitor import Monitor
from stable_baselines3.common.callbacks import CheckpointCallback,
EvalCallback

gym.register_envs(gymnasium_robotics)

ENV_ID = "HandManipulateBlockRotateZDense-v1"
START_MODEL = "models/hand_block/hand_block_checkpoint_300000_steps.
zip"

LOG_DIR = "logs/hand_block_continue"
MODEL_DIR = "models/hand_block_continue"

os.makedirs(LOG_DIR, exist_ok=True)
os.makedirs(MODEL_DIR, exist_ok=True)

def make_env(render_mode=None):
    env = gym.make(ENV_ID, render_mode=render_mode)
    env = Monitor(env)
```

```

    return env

env = make_env()
eval_env = make_env()

model = SAC.load(
    START_MODEL,
    env=env,
    device="cpu",
    replay_buffer_class=HerReplayBuffer,
    replay_buffer_kwargs=dict(
        n_sampled_goal=4,
        goal_selection_strategy="future"
    )
)

checkpoint_callback = CheckpointCallback(
    save_freq=50000,
    save_path=MODEL_DIR,
    name_prefix="continued_hand_block_checkpoint"
)

eval_callback = EvalCallback(
    eval_env,
    best_model_save_path=MODEL_DIR,
    log_path=LOG_DIR,
    eval_freq=25000,
    deterministic=True,
    render=False
)

model.learn(
    total_timesteps=300000,
    callback=[checkpoint_callback, eval_callback],
    progress_bar=True,
    reset_num_timesteps=False
)

```

```
final_path = os.path.join(MODEL_DIR, "continued_hand_block_final")
model.save(final_path)

env.close()
eval_env.close()

print(f"Saved continued model to: {final_path}.zip")
```

Listing 8: Continuing training from the best checkpoint

Run it with:

```
python continue_hand_block.py
```

To evaluate the continued models, change the model directory in the evaluation script:

```
MODEL_DIR = "models/hand_block_continue"
```

Then run:

```
python evaluate_hand_block_models.py
```

13 Using TensorBoard

TensorBoard can be used to monitor the training process.

Run:

```
tensorboard --logdir logs
```

Then open the local TensorBoard address shown in the terminal, usually:

```
http://localhost:6006
```

Important signals to watch include:

- Evaluation mean reward
- Success rate
- Actor loss
- Critic loss
- Entropy coefficient

- Episode length

For a robotic hand task, success rate is usually more important than reward alone, because the reward can improve even when the visual behavior is not ideal.

14 Finding the R1 XML File

To attach a hand to the R1 model, first locate the R1 XML file.

Run:

```
find ~/Desktop -iname "*r1*.xml"
```

In this project, the fixed model was located at:

```
/Users/cesarpascal/Desktop/Embodie AI Reports/r1_fixed/r1_fixed.xml
```

If there are multiple R1 XML files, use the fixed version first because it is more likely to load correctly in MuJoCo.

15 Inspecting R1 Body Names

Before attaching a hand, inspect the R1 body names to find the wrist body. Create `inspect_r1_bodies.py`.

```
import mujoco
import sys

if len(sys.argv) < 2:
    print("Usage:")
    print("python inspect_r1_bodies.py /path/to/r1.xml")
    raise SystemExit(1)

xml_path = sys.argv[1]

model = mujoco.MjModel.from_xml_path(xml_path)

print("\nBodies related to arm, wrist, hand, shoulder, elbow, left,
      or right:\n")

keywords = [
```

```

    "arm", "wrist", "hand", "gripper",
    "shoulder", "elbow", "forearm",
    "left", "right", "l_", "r_"
]

for i in range(model.nbody):
    name = mujoco.mj_id2name(model, mujoco.mjtObj.mjOBJ_BODY, i)

    if name is None:
        continue

    lower = name.lower()

    if any(k in lower for k in keywords):
        print(f"{i:3d}: {name}")

print("\nTotal bodies:", model.nbody)

```

Listing 9: Inspecting R1 body names

Run it with:

```
python inspect_r1_bodies.py "/Users/cesarpascal/Desktop/Embodie AI
Reports/r1_fixed/r1_fixed.xml"
```

The important output was:

```
20: left_wrist_roll_link
25: right_wrist_roll_link
```

The correct attachment body for the right hand is:

```
right_wrist_roll_link
```

16 Inspecting the Original R1 Right Wrist

The original R1 model may not have an articulated hand. It may only have simple hand geometry. To inspect it, create `inspect_right_wrist.py`.

```
import mujoco
```

```

XML_PATH = "/Users/cesarpascal/Desktop/Embodie AI Reports/r1_fixed/
r1_fixed.xml"
BODY_NAME = "right_wrist_roll_link"

model = mujoco.MjModel.from_xml_path(XML_PATH)

body_id = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_BODY,
    BODY_NAME)

if body_id == -1:
    raise RuntimeError(f"Could not find body: {BODY_NAME}")

print(f"\nBody: {BODY_NAME}")
print(f"Body id: {body_id}")

print("\nGeoms attached directly to this body:")

for geom_id in range(model.ngeom):
    geom_body_id = model.geom_bodyid[geom_id]

    if geom_body_id == body_id:
        geom_name = mujoco.mj_id2name(model, mujoco.mjtObj.
            mjOBJ_GEOM, geom_id)
        print(f"{geom_id}: {geom_name}")

print("\nJoints attached directly to this body:")

for joint_id in range(model.njnt):
    joint_body_id = model.jnt_bodyid[joint_id]

    if joint_body_id == body_id:
        joint_name = mujoco.mj_id2name(model, mujoco.mjtObj.
            mjOBJ_JOINT, joint_id)
        print(f"{joint_id}: {joint_name}")

```

Listing 10: Inspecting geoms and joints attached to the R1 right wrist

Run it with:

```
python inspect_right_wrist.py
```

The observed output was:

```
Body: right_wrist_roll_link
Body id: 25

Geoms attached directly to this body:
61: None
62: right_hand_collision

Joints attached directly to this body:
24: right_wrist_roll_joint
```

This confirms that the original R1 hand is not a real articulated hand. It is mainly geometry attached to the wrist. Therefore, replacing those geoms with a simple articulated hand is a valid first step.

17 Creating an R1 Model with a Replacement Hand

The next script removes the original wrist hand geoms and attaches a simple three-finger hand to `right_wrist_roll_link`. It also adds a visible floor and actuators for the new hand joints.

Create `make_r1_replacement_hand.py`.

```
import xml.etree.ElementTree as ET
from pathlib import Path

R1_XML = Path("/Users/cesarpascal/Desktop/Embodie AI Reports/
    r1_fixed/r1_fixed.xml")
OUT_XML = R1_XML.parent / "r1_with_replacement_hand.xml"

ATTACH_BODY_NAME = "right_wrist_roll_link"

def find_body_by_name(element, name):
    if element.tag == "body" and element.attrib.get("name") == name:
        return element

    for child in element:
        result = find_body_by_name(child, name)
```

```

        if result is not None:
            return result

    return None

tree = ET.parse(R1_XML)
root = tree.getroot()

worldbody = root.find("worldbody")
if worldbody is None:
    raise RuntimeError("Could not find <worldbody>.")

attach_body = find_body_by_name(worldbody, ATTACH_BODY_NAME)
if attach_body is None:
    raise RuntimeError(f"Could not find body named {ATTACH_BODY_NAME}")

# Remove old floor if it exists.
for geom in list(worldbody.findall("geom")):
    if geom.attrib.get("name") in ["floor", "visible_floor"]:
        worldbody.remove(geom)

# Add visible floor.
floor = ET.Element(
    "geom",
    {
        "name": "visible_floor",
        "type": "box",
        "pos": "0 0 -0.055",
        "size": "5 5 0.05",
        "rgba": "0.2 0.6 0.2 1",
        "friction": "1.0 0.5 0.5",
        "contype": "1",
        "conaffinity": "1",
    },
)
worldbody.insert(0, floor)

```

```

# Add light.
light = ET.Element(
    "light",
    {
        "name": "main_light",
        "pos": "0 -3 4",
        "dir": "0 1 -1",
        "diffuse": "1 1 1",
    },
)
worldbody.insert(1, light)

# Remove old simple hand if it already exists.
for child in list(attach_body.findall("body")):
    if child.attrib.get("name") == "simple_right_hand":
        attach_body.remove(child)

# Remove geoms directly attached to right_wrist_roll_link.
# This removes the original R1 hand collision geometry.
for child in list(attach_body):
    if child.tag == "geom":
        attach_body.remove(child)

# Simple replacement hand.
# If the hand appears in the wrong direction, tune pos and quat in
# this body line.
hand = ET.fromstring("""
<body name="simple_right_hand" pos="0.055 0 0" quat="1 0 0 0">
    <geom name="simple_right_palm" type="box" size="0.045 0.025
0.015"
        rgba="0.05 0.05 0.05 1" contype="1" conaffinity="1" mass="
0.12"/>

    <body name="simple_right_thumb_1" pos="0.010 -0.030 0">
        <joint name="simple_right_thumb_joint_1" type="hinge" axis="
0 1 0"
            range="-0.8 0.8" damping="0.2"/>

```

```

    <geom name="simple_right_thumb_1_geom" type="capsule"
        fromto="0 0 0 0.035 -0.020 0" size="0.008"
        rgba="0.2 0.2 0.2 1" mass="0.03"/>
    <body name="simple_right_thumb_2" pos="0.035 -0.020 0">
        <joint name="simple_right_thumb_joint_2" type="hinge"
            axis="0 1 0"
            range="-0.8 0.8" damping="0.2"/>
        <geom name="simple_right_thumb_2_geom" type="capsule"
            fromto="0 0 0 0.030 -0.015 0" size="0.007"
            rgba="0.2 0.2 0.2 1" mass="0.02"/>
    </body>
</body>

<body name="simple_right_index_1" pos="0.030 -0.010 0">
    <joint name="simple_right_index_joint_1" type="hinge" axis="
        0 1 0"
        range="-1.0 0.8" damping="0.2"/>
    <geom name="simple_right_index_1_geom" type="capsule"
        fromto="0 0 0 0.050 0 0" size="0.007"
        rgba="0.2 0.2 0.2 1" mass="0.03"/>
    <body name="simple_right_index_2" pos="0.050 0 0">
        <joint name="simple_right_index_joint_2" type="hinge"
            axis="0 1 0"
            range="-1.0 0.8" damping="0.2"/>
        <geom name="simple_right_index_2_geom" type="capsule"
            fromto="0 0 0 0.035 0 0" size="0.006"
            rgba="0.2 0.2 0.2 1" mass="0.02"/>
    </body>
</body>

<body name="simple_right_middle_1" pos="0.030 0.015 0">
    <joint name="simple_right_middle_joint_1" type="hinge" axis=
        "0 1 0"
        range="-1.0 0.8" damping="0.2"/>
    <geom name="simple_right_middle_1_geom" type="capsule"
        fromto="0 0 0 0.050 0 0" size="0.007"
        rgba="0.2 0.2 0.2 1" mass="0.03"/>
    <body name="simple_right_middle_2" pos="0.050 0 0">

```

```

        <joint name="simple_right_middle_joint_2" type="hinge"
            axis="0 1 0"
            range="-1.0 0.8" damping="0.2"/>
        <geom name="simple_right_middle_2_geom" type="capsule"
            fromto="0 0 0 0.035 0 0" size="0.006"
            rgba="0.2 0.2 0.2 1" mass="0.02"/>
    </body>
</body>
</body>
""")

attach_body.append(hand)

# Add actuators.
actuator = root.find("actuator")
if actuator is None:
    actuator = ET.SubElement(root, "actuator")

# Remove old simple_right actuators if they exist.
for motor in list(actuator.findall("motor")):
    if motor.attrib.get("name", "").startswith("simple_right"):
        actuator.remove(motor)

hand_motors = [
    ("simple_right_thumb_motor_1", "simple_right_thumb_joint_1"),
    ("simple_right_thumb_motor_2", "simple_right_thumb_joint_2"),
    ("simple_right_index_motor_1", "simple_right_index_joint_1"),
    ("simple_right_index_motor_2", "simple_right_index_joint_2"),
    ("simple_right_middle_motor_1", "simple_right_middle_joint_1"),
    ("simple_right_middle_motor_2", "simple_right_middle_joint_2"),
]

for motor_name, joint_name in hand_motors:
    ET.SubElement(
        actuator,
        "motor",
        {
            "name": motor_name,

```

```

        "joint": joint_name,
        "gear": "10",
        "ctrllimited": "true",
        "ctrlrange": "-1 1",
    },
)

tree.write(OUT_XML, encoding="utf-8", xml_declaration=True)

print(f"Created: {OUT_XML}")
print(f"Attached replacement hand to: {ATTACH_BODY_NAME}")

```

Listing 11: Creating an R1 XML with a simple articulated hand

Run it with:

```
python make_r1_replacement_hand.py
```

It should create:

```

/Users/cesarpascal/Desktop/Embodie AI Reports/r1_fixed/
  r1_with_replacement_hand.xml

```

18 Testing the R1 With the Replacement Hand

The R1 will fall if no standing controller is running. For hand attachment testing, gravity should be disabled. This keeps the robot still while checking if the hand is attached correctly.

Create `test_r1_with_hand.py`.

```

import time
import numpy as np
import mujoco
import mujoco.viewer

XML_PATH = "/Users/cesarpascal/Desktop/Embodie AI Reports/r1_fixed/
  r1_with_replacement_hand.xml"

model = mujoco.MjModel.from_xml_path(XML_PATH)
data = mujoco.MjData(model)

# Disable gravity for attachment testing.

```

```

# This prevents R1 from falling while inspecting the hand.
model.opt.gravity[:] = [0.0, 0.0, 0.0]

print("Model loaded successfully.")
print("XML path:", XML_PATH)
print("Number of bodies:", model.nbody)
print("Number of joints:", model.njnt)
print("Number of actuators:", model.nu)
print("Number of geoms:", model.ngeom)

print("\nChecking floor geoms:")
for i in range(model.ngeom):
    name = mujoco.mj_id2name(model, mujoco.mjtObj.mjOBJ_GEOM, i)
    if name and "floor" in name.lower():
        print(f"Found floor geom: {i} -> {name}")

print("\nChecking simple hand actuators:")
hand_actuator_ids = []

for i in range(model.nu):
    name = mujoco.mj_id2name(model, mujoco.mjtObj.mjOBJ_ACTUATOR, i)

    if name and "simple_right" in name:
        hand_actuator_ids.append(i)
        print(f"Hand actuator: {i} -> {name}")

mujoco.mj_resetData(model, data)
mujoco.mj_forward(model, data)

with mujoco.viewer.launch_passive(model, data) as viewer:
    viewer.cam.distance = 2.2
    viewer.cam.azimuth = 135
    viewer.cam.elevation = -20
    viewer.cam.lookat[:] = np.array([0.0, 0.0, 0.8])

    start_time = time.time()

    while viewer.is_running():

```

```

t = time.time() - start_time

# Move only the simple hand actuators.
for k, actuator_id in enumerate(hand_actuator_ids):
    data.ctrl[actuator_id] = 0.7 * np.sin(2.0 * t + k)

mujoco.mj_step(model, data)
viewer.sync()

```

Listing 12: Testing R1 with replacement hand and gravity disabled

Run:

```
python test_r1_with_hand.py
```

Expected result:

- The R1 should not fall because gravity is disabled.
- The original right hand collision geometry should be removed.
- A simple three-finger hand should appear near the right wrist.
- The fingers should move using sinusoidal control inputs.

19 Adjusting Hand Position and Orientation

If the hand appears on the side of the wrist, the attachment position must be adjusted. In `make_r1_replacement_hand.py`, find this line:

```
<body name="simple_right_hand" pos="0.055 0 0" quat="1 0 0 0">
```

Try these positions one at a time:

```

pos="-0.055 0 0"
pos="0 0.055 0"
pos="0 -0.055 0"
pos="0 0 0.055"
pos="0 0 -0.055"

```

After each change, regenerate and test the XML:

```

python make_r1_replacement_hand.py
python test_r1_with_hand.py

```

The correct position is the one where the palm appears where the original R1 hand was located. If the palm is in the correct location but rotated incorrectly, adjust the `quat` value.

Useful quaternion examples are:

```
quat="1 0 0 0" <!-- no rotation -->
quat="0.707 0 0 0.707" <!-- about 90 degrees around z -->
quat="0.707 0 0.707 0" <!-- about 90 degrees around y -->
quat="0.707 0.707 0 0" <!-- about 90 degrees around x -->
```

20 Troubleshooting

20.1 Deprecated Environment Error

If this error appears:

```
gymnasium.error.DeprecatedEnv: Environment version v2 for '
  HandReachDense' is deprecated.
```

Use:

```
ENV_ID = "HandReachDense-v3"
```

instead of:

```
ENV_ID = "HandReachDense-v2"
```

20.2 HER Loading Error

If this error appears:

```
AssertionError: You must pass an environment when using '
  HerReplayBuffer'
```

Load the model like this:

```
model = SAC.load(MODEL_PATH, env=env, device="cpu")
```

not like this:

```
model = SAC.load(MODEL_PATH, device="cpu")
```

20.3 R1 Falls in the Viewer

The R1 falls because no standing controller is active. During attachment testing, disable gravity:

```
model.opt.gravity[:] = [0.0, 0.0, 0.0]
```

This is only for visualization. Real training should use gravity.

20.4 No Floor Appears

If the floor is not visible, confirm that the test file is loading the correct XML:

```
XML_PATH = "/Users/cesarpascal/Desktop/Embodie AI Reports/r1_fixed/  
r1_with_replacement_hand.xml"
```

Also confirm that the terminal prints a floor geom:

```
Found floor geom: ... -> visible_floor
```

20.5 Hand Appears on the Side

The hand is attached to the correct body, but the local wrist coordinate direction may not match the expected direction. Tune this line:

```
<body name="simple_right_hand" pos="0.055 0 0" quat="1 0 0 0">
```

Test different `pos` values until the palm is located correctly.

21 Recommended Next Steps

The current R1 hand integration is a testing model. The next steps are:

1. Align the simple hand perfectly with the R1 right wrist.
2. Replace the simple hand with a more realistic hand model, such as a LEAP-style hand or another MJCF-compatible robotic hand.
3. Create a new MuJoCo environment where the R1 arm and hand reach for an object.
4. Train a simple reaching task first.
5. Add grasping by placing a ball or cube near the hand.

6. Add lifting or rotation after reaching and grasping works.
7. Eventually combine the arm-and-hand task with full-body R1 balance or locomotion.

Do not start with full-body walking plus dexterous manipulation at the same time. That is too difficult for a CPU-only setup. The best strategy is to solve the hand task first, then the arm task, and finally combine the behavior with the rest of the robot.

22 Final Checklist

A working setup should satisfy the following checklist:

- The hand environments are listed successfully.
- `HandReachDense-v3` opens in the MuJoCo viewer.
- The reach policy trains and saves a model.
- The reach policy can be replayed.
- `HandManipulateBlockRotateZDense-v1` trains with SAC and HER.
- All saved checkpoints are evaluated.
- The best checkpoint is selected using success rate and reward.
- The R1 body list shows `right_wrist_roll_link`.
- The original right wrist geoms are identified.
- `r1_with_replacement_hand.xml` is created.
- The R1 model loads with the replacement hand.
- Gravity is disabled during attachment testing so the robot does not fall.
- The simple hand fingers move in the viewer.

23 Conclusion

This manual provides the complete workflow for training a MuJoCo robotic hand and beginning its integration with the Unitree R1 model. The standalone Shadow Hand training demonstrates that the hand can learn both reaching and block manipulation. The checkpoint evaluation process shows why success rate and visual behavior are important when selecting the best model. The R1 integration process shows how to inspect the robot XML, identify the right wrist body, remove the original hand collision geometry, and attach a simple articulated hand. This creates a foundation for future work involving R1 arm control, grasping, object manipulation, and eventually full-body robotic tasks.